**THE STATE UNIVERSITY OF ZANZIBAR**

**SCHOOL OF COMPUTING, COMMUNICATION AND MEDIA STUDIES**

**DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY**

# SOFTWARE DESIGN DOCUMENT

## Designing and Developing a Warehouse Management System for Fumba Port Indoor Storage Facilities

**Final Year Project • Bachelor of Science in Computer Science**
Academic Year 2025/2026
Version 1.0 • 1 September 2026

*Prepared for academic assessment and system implementation reference*

# DOCUMENT CONTROL

| Item | Value |
|---|---|
| Document title | Software Design Document – Fumba Port Warehouse Management System |
| System short name | Fumba Port WMS |
| Version | 1.0 |
| Status | Implementation-aligned baseline |
| Prepared for | The State University of Zanzibar – Final Year Project |
| Date | 1 September 2026 |
| Configuration baseline | Repository implementation and database migrations available on the document date |

## Approval Record

| Role | Responsibility | Approval |
|---|---|---|
| Project author | Document preparation and implementation traceability | To be signed |
| Project supervisor | Academic and technical review | To be signed |
| User representative | Operational suitability review | To be signed |

## Revision History

| Version | Date | Description |
|---|---|---|
| 1.0 | 1 Sep 2026 | Initial complete SDD aligned with the implemented Fumba Port WMS. |

# OVERVIEW

This document defines the architecture, data design, user interfaces, module logic, integrations and integrity controls of the Fumba Port Warehouse Management System. It converts the supplied SDD template into an implementation-specific design baseline for development, testing, deployment and academic assessment.

**Scope statement:** The document describes the repository implementation as of 1 September 2026. Cloud hosting, production SMTP operation and live-payment credentials are outside the demonstrated deployment baseline.

# TABLE OF CONTENTS



Word users can update page numbering from the References tab if the document is expanded.

# 1. INTRODUCTION

## 1.1 Purpose and Scope

The purpose of this Software Design Document (SDD) is to define how the Fumba Port WMS fulfils the operational need for secure, traceable and capacity-aware indoor freight handling. The design covers the browser user interface, REST and real-time backend, PostgreSQL persistence, Docker deployment, role-based authorization and Flutterwave payment integration.

In scope are cargo registration and review; configurable warehouse hierarchy; rule-based bin recommendation; scanner-assisted placement; customs inspection; tariff, invoice and payment processing; management release; gate-out; notifications, reporting, configuration backup and immutable audit trails. Out of scope are vessel scheduling, outdoor yard planning, physical PLC/conveyor control, customs-government data exchange and public-cloud production operations.

## 1.2 Project Executive Summary

Manual freight tracking at an indoor port warehouse creates risks of lost records, over-capacity storage, delayed clearance, billing errors and unauthorized release. The WMS provides one controlled digital workflow from receipt to dispatch. Cargo is registered against an effective tariff, reviewed by a supervisor, placed only in eligible capacity, cleared by customs, financially settled or management-released, and dispatched only after a final readiness decision.

### 1.2.1 System Overview

Users access role-specific React portals through a browser or handheld scanner. Nginx terminates client connections in production and forwards API and Socket.IO traffic to an Express application. Business services enforce workflow and financial rules and persist transactional state in

PostgreSQL. Flutterwave is the only external transactional service in the current design; SMTP is optional for payment notifications.

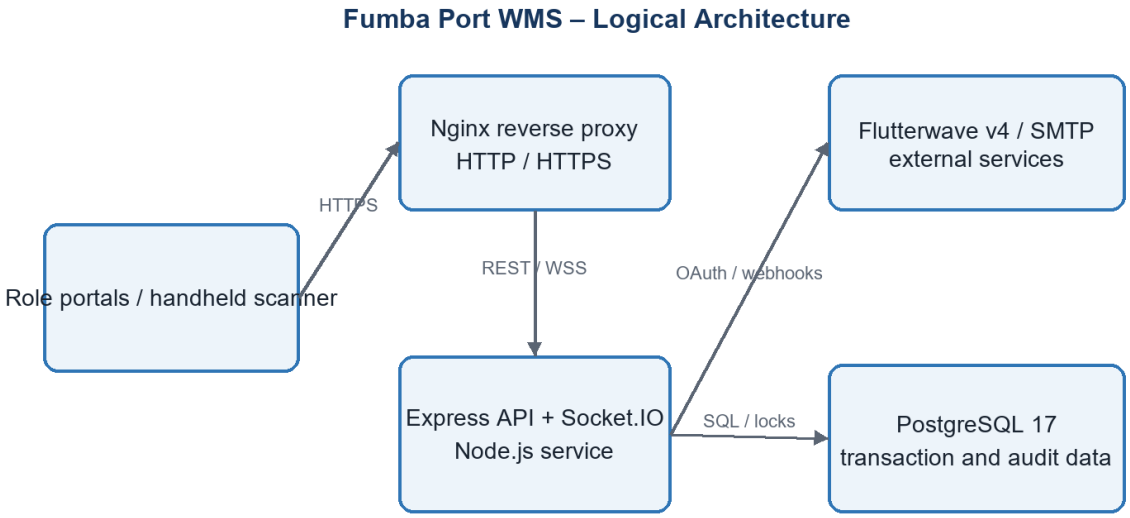


*Figure 1. High-level logical architecture*

### 1.2.2 Design Constraints

- The demonstration environment is a single-host Docker Compose deployment bound to localhost by default.
- PostgreSQL is the authoritative data store; workflow state transitions must be transactional and concurrency-safe.
- Financial values must use database numeric types and fixed-point application arithmetic, never binary floating point.
- The system must support nine operational identities with least-privilege permissions and warehouse/shift scoping.
- Cargo release must not bypass customs clearance or physical placement; management release may override payment only.
- Uploaded documents are limited to allowed formats and validated using file signatures, not extensions alone.

### 1.2.3 Assumptions and Future Contingencies

The design assumes reliable local connectivity, barcode-capable scanners, trained role holders and correctly configured tariffs, warehouse hierarchy and permissions. Future growth may require managed PostgreSQL, object storage for cargo documents, a Redis-backed Socket.IO adapter, centralized observability, multi-site tenancy and a formal customs interface. The modular service and route boundaries allow those changes without replacing the user interface or domain workflow.

## 1.3 Document Organization

Section 2 defines architecture; Section 3 defines persistent and file data; Section 4 defines operator interaction; Section 5 specifies module behavior; Section 6 describes external interfaces; Section 7 defines security and integrity controls. Appendices provide a compact data dictionary, traceability matrix and deployment baseline.

## 1.4 Points of Contact

| Contact role | Area |
|---|---|
| Project author / developer | Design, implementation, test evidence and configuration |
| Academic supervisor | Research method, academic quality and acceptance |
| Warehouse operations representative | Cargo handling, placement and dispatch validation |
| Finance and customs representatives | Tariff/payment and inspection/clearance validation |

## 1.5 Project References

- Repository README and source code baseline.
- backend/database/schema.sql and dated SQL migrations.
- Backend route, controller, service and automated-test suites.
- Docker Compose, Nginx and HTTPS deployment configuration.
- Flutterwave v4 OAuth/payment and webhook integration implemented by the project.

## 1.6 Glossary

| Term | Meaning |
|---|---|
| API | Application Programming Interface |
| CRUD | Create, Read, Update and Delete |
| DBMS | Database Management System |
| FRD/SRS | Functional/Software Requirements Document |
| HMAC | Hash-based Message Authentication Code |
| JWT | JSON Web Token |
| RBAC | Role-Based Access Control |
| RTM | Requirements Traceability Matrix |

| Term | Meaning |
| --- | --- |
| SDD | Software Design Document |
| WMS | Warehouse Management System |
| WSS | Secure WebSocket communication |

# 2. SYSTEM ARCHITECTURE

## 2.1 Architectural Style and Principles

The system uses a three-tier web architecture with a componentized single-page application, a stateless HTTP API plus stateful real-time channel, and a relational database. Controllers handle protocol concerns; services own domain rules; configuration registries provide policy authority; SQL constraints and transactions provide last-line integrity.

- Single source of truth for workflow and authorization policy.
- Defence in depth across browser, API middleware, service logic and database constraints.
- Explicit public references for user-facing identities while internal keys remain private.
- Idempotent handling of payment webhooks and replay-resistant refresh-token rotation.
- Configuration changes are validated, audited and recoverable through backup/restore.

## 2.2 System Hardware Architecture

| Component | Minimum / recommended role | Connectivity |
|---|---|---|
| Application host | 64-bit dual-core CPU, 8 GB RAM, 20 GB free storage; Docker-capable OS | LAN; optional Internet for payment/email |
| Operator workstation | Modern browser, 1366×768 or higher | HTTPS to application host |
| Handheld scanner | Barcode reader with browser and Wi-Fi; camera or keyboard-wedge input | HTTPS/WSS over trusted WLAN |
| Barcode/office printer | Standard label or A4 printer | Operator workstation / LAN |
| Backup storage | Encrypted external or network storage sized to database and document retention | Scheduled administrative access |

## 2.3 System Software Architecture

| Layer / component | Technology | Responsibility |
|---|---|---|
| Presentation | React 18, Vite, React Router, React Query, Radix/Tailwind | Role portals, forms, dashboards, responsive interaction and server-state caching |
| API edge | Nginx 1.27 | Static delivery, reverse proxy, security headers and TLS termination |
| Application | Node.js 18+, Express 4 | REST endpoints, validation, authentication and response handling |

| Layer / component | Technology | Responsibility |
|---|---|---|
| Domain services | JavaScript service modules | Cargo workflow, placement, finance, readiness, notifications and configuration |
| Real time | Socket.IO 4 | Scanner sessions and operational event broadcasts |
| Persistence | PostgreSQL 17 + pgcrypto | Transactions, constraints, indexed records, locks and immutable audit data |
| Infrastructure | Docker / Compose | Repeatable development and production-like deployment |

### 2.3.1 Backend Module Decomposition

| Module | Key responsibilities |
|---|---|
| Authentication and sessions | Login, refresh-family rotation, logout, password changes and scanner-account isolation. |
| Cargo and approvals | Registration, duplicate checks, documents, supervisor decision and correction/resubmission. |
| Warehouse hierarchy | Warehouses, zones, racks, levels, bins, shifts, capacity and assignment history. |
| Placement and scanning | Rule evaluation, recommendation, validation, scan pairing, placement confirmation and overrides. |
| Customs | Inspection start, holds, status transitions, clearance and history. |
| Finance and payments | Tariffs, approval, draft/issued invoices, payment recording, Flutterwave initiation and webhook settlement. |
| Release and gate | Readiness evaluation, management/emergency decisions and final dispatch. |
| Governance | RBAC, audit, notifications, reports, readiness, configuration backup/restore. |

## 2.4 Internal Communications Architecture

| Path | Protocol / format | Design behavior |
|---|---|---|
| Browser ↔ Nginx | HTTPS; JSON; static assets | Same-origin production access; cacheable frontend assets. |
| Nginx ↔ backend | HTTP/1.1; WebSocket upgrade | Routes `/api` and Socket.IO traffic to the application service. |
| Backend ↔ PostgreSQL | TCP; PostgreSQL wire protocol | Parameterized SQL, pooled connections, transactions and advisory/row locks. |
| Browser ↔ Socket.IO | WSS events | Authenticated role/user rooms; scanner session feedback and operational updates. |

| Path | Protocol / format | Design behavior |
|---|---|---|
| Container network | Docker bridge DNS | Services use stable Compose service names; database is not public in production. |

**Communication rule:** REST is authoritative for commands and queries. Socket.IO carries notifications and live status only; clients reconcile with REST after reconnecting.

# 3. FILE AND DATABASE DESIGN

## 3.1 Database Management System Files

PostgreSQL stores all authoritative configuration, identity, cargo, financial, inspection, dispatch and audit data. The schema is created from schema.sql and evolved by ordered migrations recorded in schema_migrations. Foreign keys preserve relationships; check constraints restrict states; indexes support queues and reporting; transactions protect multi-record workflow changes.

### 3.1.1 Logical Data Model

| Domain | Principal tables | Relationship summary |
|---|---|---|
| Identity and access | roles, permissions, role_permissions, users, user_sessions, scanner_accounts, scanner_sessions | Users belong to a role and optional warehouse/shift; sessions and scanner pairing are independently revocable. |
| Storage topology | warehouses, zones, racks, levels, bins, capacity configurations | Strict warehouse→zone→rack→level→bin hierarchy with weight/volume capacity and status. |
| Cargo workflow | cargo, cargo_documents, approval_requests, cargo_approval_history, cargo_movements, cargo_locations | Cargo owns documents and histories; current location is separately tracked and audited. |
| Placement | bin_rules, placement_validation_logs, barcode_print_logs, bin_barcode_print_logs | Configurable rules determine eligible bins; every validation and print operation is recorded. |
| Finance | storage_tariffs, tariff_approval_requests, invoices, invoice_items, payments, payment_attempts, payment_webhook_events | Cargo is billed by an approved tariff; invoice and payment lifecycles preserve ledger history and idempotency. |
| Release and governance | management_release_requests, dispatch_requests, notifications, audit_logs, archived_audit_logs, system_settings | Release decisions, operator alerts, configuration and immutable evidence are retained. |

### 3.1.2 Key Data Rules and Access Methods

- Cargo identity is protected by partial unique indexes covering active delivery-note, container and vehicle/consignee/type combinations.
- Queue access uses composite indexes on registration, placement, approval, dispatch and notification status with descending creation dates.
- Foreign keys bind cargo to warehouses, users, locations and workflow records; service transactions lock mutable rows before decisions.
- Audit tables deny UPDATE, DELETE and TRUNCATE privileges to the runtime user; archives preserve historical records.

- The expected data volume is moderate for a single facility. Indexes support online transaction processing; archival and backup policies manage long-term growth.

### 3.1.3 Transaction Boundaries

| Transaction | Atomic effects |
|---|---|
| Cargo registration | Validate configuration and tariff; insert cargo; create draft invoice; create audit/notification records. |
| Supervisor approval | Lock cargo/invoice; record decision; issue invoice and public reference; queue customer notification. |
| Placement confirmation | Lock cargo/bin; re-evaluate rules and capacity; write location/movement/log; update occupied capacity. |
| Payment settlement | Verify invoice and idempotency key; write payment/attempt event; update totals and status. |
| Gate-out | Re-evaluate release readiness; lock cargo; record dispatch; set placement state to Dispatched; audit actor. |

## 3.2 Non-DBMS Files

| File class | Purpose | Readers / writers | Control |
|---|---|---|---|
| Cargo documents | Operator-uploaded supporting evidence | Cargo module; authorized reviewers | Magic-byte/type/size validation; generated safe name; access-controlled retrieval |
| Configuration backup | Portable administrative snapshot | System administrator | Schema and content validation before restore; operation audit and rate limit |
| Report export | CSV/PDF-like downloadable operational output | Authorized finance, management and role-report users | Generated on demand; no authoritative state |
| Application logs | Operational diagnosis and email fallback | Backend runtime / administrator | Sensitive-value redaction; environment-controlled retention |
| Frontend assets | Compiled HTML, CSS, JavaScript and images | Nginx / browsers | Immutable build output and cache policy |

# 4. HUMAN-MACHINE INTERFACE

## 4.1 Interaction Model

The interface presents a landing/login flow followed by role-specific portals. A shared application layout provides navigation, authenticated identity, notifications and logout. Permissions from the backend hide or disable actions, but server authorization remains decisive. Destructive or workflow-changing operations use confirmation dialogs and require notes where accountability is needed.

## 4.2 Inputs

| Input / screen | Primary users | Core data and validation |
|---|---|---|
| Login and initial setup | All roles / initial administrator | Username/password; rate limit; one-time bootstrap lock; password policy. |
| Cargo registration | Warehouse staff | Consignee, delivery/container/vehicle identifiers, cargo type, dimensions, weight, hazard/fragility, warehouse, documents; mandatory and duplicate validation. |
| Supervisor review | Supervisor | Approve, reject or request correction; decision notes required for adverse/exception outcomes. |
| Scanner placement | Staff and paired scanner | Cargo barcode then bin barcode; active session, assignment, capacity, rule and status validation. |
| Customs processing | Customs officer | Inspection start, clearance/hold status and notes; authorized transitions only. |
| Tariff and payment | Finance / customer | Rate basis, effective dates, approval; payment initiation through time-bound public token; idempotent confirmation. |
| Release / gate-out | Management / gate | Release request reasons, decision notes, cargo barcode/reference; readiness rechecked at confirmation. |
| System configuration | Administrator | Hierarchy, capacity, roles, permissions, forms, notifications and backup; validation and audit required. |

### 4.2.1 Input Control Rules

- Client-side Zod/form checks provide immediate guidance; API validation rejects bypassed or malformed requests.
- Identifiers, numeric bounds, state enums, object depth and property counts are constrained.
- Document content is validated by binary signature and allowed MIME/type policy.
- Permission and warehouse scope are derived from authenticated context, not accepted from untrusted client claims.

- Barcode operations bind to a live scanner session and server-side cargo/bin records.

## 4.3 Outputs

| Output | Audience | Purpose and access |
|---|---|---|
| Role dashboards | Each operational role | Queues, counts, alerts and next actions limited by permission and warehouse scope. |
| Barcode labels | Warehouse staff | Machine-readable cargo/bin identifiers with human-readable reference; print event logged. |
| Invoice and payment link | Finance/customer | Itemized charges, paid amount, balance and approved online-payment action. |
| Placement recommendation | Warehouse staff/supervisor | Ranked eligible bins and rule explanations; recommendation is revalidated at placement. |
| Customs/release history | Customs, gate, supervisor, auditor | Chronological evidence supporting clearance and release decisions. |
| Operational and financial reports | Management, finance, auditor | Filtered summaries and exports controlled by report permissions. |
| Notifications | Authorized recipients | Pending reviews, escalations, announcements and workflow events via portal/realtime channel. |
| Audit log | Administrator/auditor | Actor, time, network context, action, target and change metadata. |

## 4.4 Accessibility and Usability

Interfaces use semantic labels, keyboard-capable component primitives, visible focus states, descriptive feedback and responsive layouts. Status is communicated through text as well as color. Tables provide headings and filters; dialogs keep action labels explicit. The production acceptance test should include keyboard-only use, contrast review, screen-reader labels and handheld viewport testing.

# 5. DETAILED DESIGN

## 5.1 Hardware Detailed Design

The application does not directly control industrial hardware. It uses standard computers, printers and browser-capable scanners. Production procurement should select devices that satisfy the following logical requirements.

| Item | Detailed requirement |
|---|---|
| Server | 64-bit CPU, at least 8 GB RAM, SSD storage, reliable clock synchronization, daily backup target and UPS protection. |
| Client | Current Chromium/Firefox/Edge browser, 1366×768 display or better, keyboard and pointing device. |
| Scanner | Reads configured 1D/2D barcode format; injects text or exposes browser camera; stable Wi-Fi; protected user session. |
| Printer | Supports the selected cargo/bin label size and A4 invoice/report output; client OS driver supplied by vendor. |
| Network | Trusted LAN/WLAN, TLS-capable edge, firewall allowing only required application ports; Internet egress for Flutterwave/SMTP. |

## 5.2 Software Detailed Design

### 5.2.1 Authentication and Authorization

The login controller validates credentials with bcrypt and issues a short-lived JWT plus database-backed refresh session. Refresh tokens rotate by family; reuse of a superseded token indicates replay and revokes the affected family. Middleware authenticates the request, distinguishes scanner accounts and checks a centralized permission registry. Services additionally apply warehouse/role scope to data queries.

### 5.2.2 Cargo Registration and Review

- Load the published registration-form configuration and effective, management-approved tariff.
- Validate cargo fields, dimensions, weights, identifiers, warehouse assignment and uploaded evidence.
- Check active duplicate identities and insert cargo in Pending Review state.
- Create a Draft invoice without a payment reference and write the audit trail.
- Supervisor approves, rejects or requests correction. Approval issues the existing invoice; rejection cancels payment capability while preserving history.

### 5.2.3 Placement Rule Engine

The bin-rule engine obtains enabled rule definitions from configuration, resolves each evaluator from an allow-listed registry and evaluates candidate bins against cargo and topology context. Typical evaluators include remaining weight/volume, cargo type, hazardous classification, fragility, customs conditions and status. Recommendations contain eligible locations and reasons; confirmation repeats evaluation inside a transaction to prevent stale-capacity placement.

**Concurrency:** Placement locks the affected cargo and bin/capacity rows before writing the current location and occupied capacity. A recommendation alone never reserves capacity.

### 5.2.4 Cargo State Machine

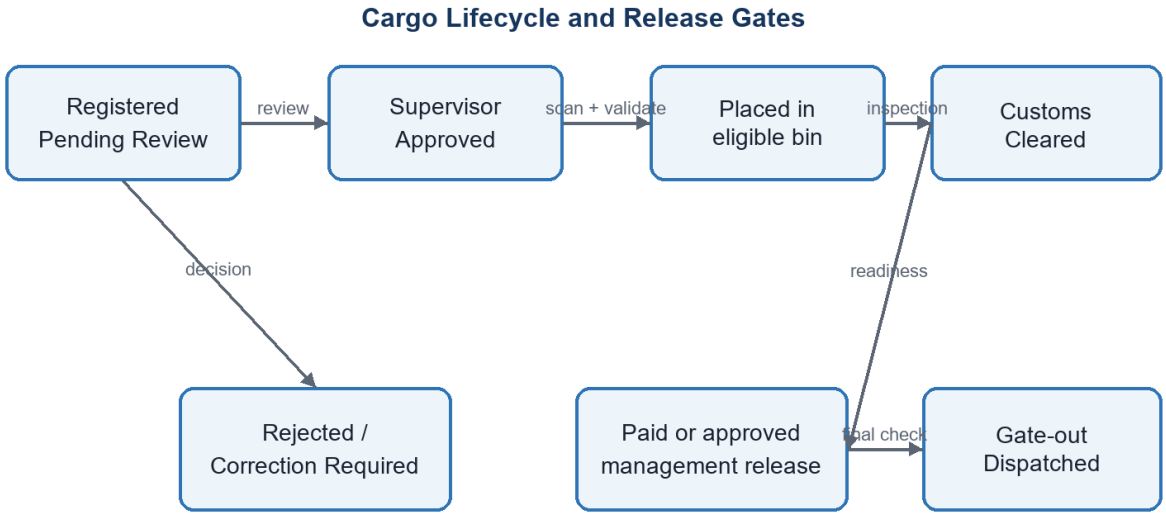


*Figure 2. Cargo workflow and mandatory release gates*

### 5.2.5 Customs, Finance and Release

Customs operations record inspection history and a current clearance/hold status. Finance calculates charges with scaled BigInt arithmetic, manages tariff approval and invoice lifecycles, and accepts manual or Flutterwave settlement. Release readiness is a pure policy evaluation over registration approval, physical placement, customs clearance, invoice settlement or approved management release, and absence of blocking state. Gate-out repeats the evaluation transactionally and records the dispatch actor and time.

### 5.2.6 Notifications, Reports and Audit

Domain events create recipient-scoped notifications according to configurable policies and escalation settings. Role reports aggregate only authorized scopes and export through dedicated endpoints. Audit middleware and services capture user, role/warehouse snapshots, IP or terminal

context, action, target, timestamp and metadata. Runtime database grants make audit records append-only.

### 5.2.7 Error Handling and Logging

Controllers pass failures to a common error middleware that converts known API errors to minimized JSON responses and prevents stack or secret leakage. Request identifiers and structured logs support diagnosis. Validation failures use 4xx responses; authorization uses 401/403; conflicts such as duplicate or stale state use 409; unexpected errors use 500 and are logged server-side.

## 5.3 Internal Communications Detailed Design

| Concern | Detailed design |
|---|---|
| API format | JSON request/response under `/api`; success payloads contain data/count where applicable; errors are minimized. |
| Authentication | Bearer access token; refresh endpoint uses a rotatable server-tracked session token; scanner credentials are isolated. |
| Real-time events | Socket.IO authenticated connection joins permitted user/role/session rooms; events signal change and clients refresh authoritative data. |
| Database access | `pg` connection pool; parameterized statements; explicit BEGIN/COMMIT/ROLLBACK for multi-write operations. |
| Timeout/retry | External payment calls use bounded failure handling; webhook retries are safe because event/provider references are idempotent. |

# 6. EXTERNAL INTERFACES

## 6.1 Interface Architecture

External interfaces are isolated behind backend services. Browsers never receive provider credentials or connect directly to the database. Outbound payment and email calls originate from the backend. Inbound payment callbacks terminate at a dedicated webhook route and are authenticated before any financial state changes.

## 6.2 Interface Detailed Design

| Interface | Direction and protocol | Data / handshake | Failure handling |
|---|---|---|---|
| Flutterwave OAuth/payment | Backend ↔ provider over HTTPS | Client credentials obtain access token; payment request carries amount, currency, reference and return context. | Reject unavailable/invalid provider response; persist attempt state; show safe user message. |
| Flutterwave webhook | Provider → backend HTTPS POST | Raw payload plus configured HMAC-SHA256 signature; provider event/reference identifies transaction. | Reject invalid signature; record event idempotently; duplicate delivery returns without duplicate payment. |
| SMTP | Backend → mail server using TLS as configured | Payment-link and notification message to validated recipient. | Delivery failure is logged/queued; when SMTP is absent the demonstration uses logger fallback. |
| Browser client | Browser ↔ Nginx over HTTPS/WSS | JSON API and Socket.IO frames; access token and server permission checks. | Standard 4xx/5xx response; reconnect then REST reconciliation for real-time channel. |

### 6.2.1 Public Payment Interface

The customer opens `/pay/:token`. The server resolves a high-entropy token to an eligible issued invoice and exposes only the summary necessary to pay. Initiation is rate limited and records a payment attempt. A token is not an authentication credential for finance administration and cannot access cargo documents, internal notes or unrelated invoices.

### 6.2.2 Webhook Processing Sequence

- Capture the exact request body and required provider signature header.
- Compute HMAC using the configured webhook secret and compare safely.
- Validate event type, provider reference, currency, amount and expected invoice relationship.

- Insert/lock the webhook event using its unique identity; if already processed, return success without reapplying.
- Write payment/ledger changes and invoice status in one transaction; emit audit/notification after commit.

# 7. SYSTEM INTEGRITY CONTROLS

## 7.1 Security Control Objectives

| Objective | Implemented design control |
|---|---|
| Identification and authentication | bcrypt password hashes; short-lived JWT; refresh-session rotation, revocation and replay detection; one-time bootstrap lock. |
| Least privilege | Central permission registry, route middleware, non-scanner restrictions and warehouse/shift data scoping. |
| Confidentiality | TLS at the production edge; secret environment variables; response minimization; token/password redaction. |
| Integrity | Schema constraints, parameterized SQL, transactions, locks, fixed-point money arithmetic and idempotent provider events. |
| Availability | Container health/restart strategy, database persistence, backup/restore, bounded rate limits and operational readiness checks. |
| Accountability | Append-only audit data containing actor, role/warehouse snapshot, time, network context, target and metadata. |

## 7.2 Input, Session and API Controls

- Request validation rejects excessive nesting/property counts and prototype-pollution keys.
- Uploads are constrained by type, size and binary signature; retrieval requires authorization.
- Authentication, refresh, bootstrap, public payment and administrative operations have purpose-specific rate limits.
- Security headers, restrictive CORS and production HTTPS reduce browser attack surface.
- Responses exclude hashes, raw tokens, provider secrets and avoid revealing internal error stacks.

## 7.3 Workflow and Financial Integrity

Every critical transition is authorized, state-checked and audited. Cargo cannot be placed before approval, capacity cannot become negative, hazardous cargo cannot enter an ineligible bin, customs holds block release, and an unpaid invoice requires a separately approved management release.

Management release does not override customs or placement. Gate-out is the final transactional enforcement point.

## 7.4 Audit, Retention and Recovery

Audit records are immutable to the runtime account and can be archived only through an authorized, rate-limited administrative workflow. Configuration backup includes validation before restore. Production operation should add scheduled PostgreSQL backups, encrypted off-host copies, restore drills, log retention thresholds and documented recovery time/recovery point objectives.

## 7.5 Verification Strategy

| Verification level | Evidence |
|---|---|
| Unit and service | Node test suites for rule evaluators, workflow, finance, notification, configuration and security logic. |
| API integration | Tests for RBAC, route contracts, payment/webhook behavior, customs/gate policies and concurrency. |
| Frontend component | Vitest and Testing Library tests for portals, access rules, scanner/payment flows and critical components. |
| Database | Schema verification, ordered migrations, constraints/grant checks and PostgreSQL typing tests. |
| Release/UAT | Readiness service, Docker build, role-based end-to-end scenarios and localhost user acceptance testing. |

# APPENDIX A. COMPACT DATA DICTIONARY

| Entity | Representative elements | Validation / maintenance |
|---|---|---|
| users | public reference, username, password hash, role, warehouse, shift, status | Unique identity; administrator-managed; self-profile changes limited. |
| cargo | reference, consignee, delivery/container/vehicle IDs, type, weight, volume, hazard/fragility, registration/customs/placement states | Required fields and enums; duplicate checks; workflow services update. |
| bins | barcode, hierarchy keys, weight/volume capacity, status, special attributes | Unique within level; capacity and eligibility protected during placement. |
| approval_requests | cargo, type, status, requester/assignee, decision notes/times | Created by workflow; authorized decision; history retained. |
| invoices | number, cargo, tariff, subtotal/total, paid amount, status, public payment token | Unique number/token; fixed-point calculation; lifecycle restricted. |
| payments | reference, invoice, amount, method/provider, status, confirmed time | Positive amount; idempotent provider reference; finance/provider updates. |
| audit_logs | actor/snapshots, action, target, network context, metadata, time | Append-only runtime access; archive through controlled operation. |
| notifications | recipient user/role/warehouse, event type, priority, read/archive/resolve state | Policy-driven creation; user-scoped maintenance. |

# APPENDIX B. REQUIREMENTS TRACEABILITY MATRIX

| ID | Requirement | Design allocation | Verification |
|---|---|---|---|
| FR-01 | Register and review cargo | 5.2.2; cargo/approval modules | Cargo workflow and supervisor tests |
| FR-02 | Prevent storage over-allocation | 3.1.3; 5.2.3 | Placement, capacity and concurrency tests |
| FR-03 | Barcode-assisted placement | 4.2; 5.2.3 | Scanner session and placement tests |
| FR-04 | Customs inspection and hold | 5.2.5 | Customs workflow/authority tests |
| FR-05 | Tariff, invoice and payment | 3.1; 5.2.5; 6.2 | Finance and Flutterwave tests |
| FR-06 | Controlled cargo release | 5.2.4–5.2.5 | Readiness, management and gate tests |
| FR-07 | Role-restricted portals | 2.3; 7.1–7.2 | RBAC and portal-access tests |
| FR-08 | Immutable audit and reporting | 5.2.6; 7.4 | Audit grants/log access/report tests |

| ID | Requirement | Design allocation | Verification |
|---|---|---|---|
| NFR-01 | Security and privacy | 7.1–7.4 | Security-hardening tests and review |
| NFR-02 | Reliability under concurrency | 3.1.3; 5.2.3 | Final concurrency validation |

# APPENDIX C. DEPLOYMENT AND OPERATIONS

| Service | Development binding | Container port | Dependency |
|---|---|---|---|
| Frontend | 127.0.0.1:3000 | 3000 | Backend API |
| Backend | 127.0.0.1:5000 | 5000 | PostgreSQL; optional Flutterwave/SMTP |
| PostgreSQL | 127.0.0.1:5433 | 5432 | Persistent volume |

## C.1 Deployment Procedure

- Create the environment file from the supplied example and replace all secrets.
- Build and start the Docker Compose services.
- Apply database migrations and verify schema/grants.
- Run automated backend and frontend tests.
- Complete one-time administrator setup and configure roles, hierarchy, tariffs and notification policies.
- Execute role-based UAT for registration, placement, customs, payment and gate-out.
- For production, configure DNS/TLS, backups, SMTP, provider credentials and monitoring before exposing the service.

## C.2 Known Limitations and Planned Enhancements

- Current baseline targets a local single-instance demonstration, not a public multi-node deployment.
- Email falls back to application logging when SMTP is not configured.
- External customs integration and object storage are not implemented.
- Production readiness requires penetration testing, backup restoration evidence, accessibility review and live-provider certification.